SOFTWARE CHANGE

IMPLEMENTATION REPORT

CR-004: Student Input Field Validation for Add and Edit Operations

Field	Details
Change Request ID	CR-004
Change Request Title	Student Input Field Validation for Add and Edit Operations
Student Name	T.Lakshika
Student ID	JF/ICT/24/32
Implementation Status	Completed
Date	8 October 2026

1. Change Request Summary

1.1 Business Requirement

Administrators previously could enter student information without comprehensive validation when adding or editing students.

For example, invalid phone numbers, short names, invalid email addresses, and weak passwords could potentially be entered.

The change was required to improve student data quality and account security by validating student information before it is accepted.

The system must validate registration number, first name, last name, email address, phone number, and password according to the defined requirements.

Both client-side and server-side validation are required so that invalid information cannot be submitted through the user interface or directly through an API request.

1.2 Existing Behaviour

Before implementation:

- `StudentFormModal.jsx` performed only basic required-field validation.
- First name and last name did not have a minimum 3-character requirement.

- Invalid numbers or inappropriate special characters could be entered into name fields.
- Email validation was not comprehensive.
- Phone numbers were not strictly limited to exactly 10 numeric digits.
- New student passwords did not enforce the complete strong password policy.
- Backend validation did not comprehensively validate all student fields.
- Invalid data could potentially be submitted directly to the backend.
- Existing Add, Edit, and Delete operations used the existing student ID.
- Existing administrator authentication and authorization were already implemented.

1.3 Requested Change

Comprehensive validation was required for both Add Student and Edit Student operations.

The implementation must:

- Validate all required student fields.
- Validate the registration number.
- Require first and last names to contain at least 3 characters.
- Allow valid alphabetic name characters.
- Reject invalid numbers and inappropriate special characters in names.
- Validate email address format.
- Require exactly 10 digits for phone numbers.
- Allow numeric characters only in phone numbers.
- Require a password when creating a new student.
- Require passwords to contain at least 8 characters.
- Require uppercase and lowercase letters.
- Require at least one number.
- Require at least one special character.
- Display clear validation error messages.
- Prevent invalid frontend submissions.
- Apply the same critical validation on the backend.
- Prevent client-side validation bypass.
- Preserve existing Add, Edit, and Delete functionality.
- Preserve administrator authentication and authorization.
- Continue securely hashing student passwords.

2. Implementation

2.1 Implementation Summary

CR-004 was implemented as a frontend and backend validation enhancement for the administrator's student management functionality.

Client-side validation was added to `StudentFormModal.jsx`. The form now validates student information before an API request is sent.

Server-side validation was added to `studentController.js`. The same critical validation rules are applied on the backend so that invalid data cannot bypass the frontend through direct API requests.

The implementation validates:

- Registration number
- First name
- Last name
- Email
- Phone number
- Password during student creation

The existing API endpoints, database structure, authentication, authorization, Add, Edit, and Delete operations remain unchanged.

2.2 Affected Files

File	Change	Purpose
`frontend/src/components/StudentFormModal.jsx`	Modified	Added client-side validation for Add and Edit operations
`backend/controllers/studentController.js`	Modified	Added server-side validation for student input
`backend/models/studentModel.js`	No change	Existing database operations remain unchanged
`backend/routes/studentRoutes.js`	No change	Existing API routes and authorization remain unchanged
`database/schema.sql`	No change	No database schema modification was required
Existing student form stylesheet	Verified	Existing form/error styling used

Additional Files

No additional files were required for CR-004.

2.3 Technical Changes

Client-Side Validation

Validation was added to `StudentFormModal.jsx` before submitting the form.

A dedicated validation function was implemented to check the entered values before the API request is generated.

The validation prevents invalid information from being submitted through the administrator interface.

Registration Number Validation

The registration number:

- Must not be empty.
- Must contain valid characters.
- Supports values such as:

```
JF/ICT/26/001
JF-ICT-26-001
```

Invalid characters are rejected.

First Name Validation

The first name:

- Must not be empty.
- Must contain at least 3 characters.
- Must contain valid alphabetic characters.
- Numbers are rejected.
- Invalid special characters are rejected.

Examples:

```
Saman       → Valid
John        → Valid
Jo          → Invalid
John123     → Invalid
```

Last Name Validation

The last name follows the same rules:

- Required.
- Minimum 3 characters.
- Alphabetic characters are allowed.
- Numbers are rejected.
- Inappropriate special characters are rejected.

Email Validation

The email address is checked using an email format validation pattern.

Example:

```
student@gmail.com       → Valid
student.example.com     → Invalid
student@gmail          → Invalid
```

Phone Number Validation

The phone number:

- Must be required.
- Must contain numeric characters only.
- Must contain exactly 10 digits.
- Fewer than 10 digits are rejected.
- More than 10 digits are rejected.

Example:

```
0712345678       → Valid
071234567        → Invalid
07123456789      → Invalid
07123abc78       → Invalid
```

The frontend also uses:

```
maxLength = 10
inputMode = numeric
```

to improve user input handling.

Strong Password Validation

For new student creation, the password must:

- Be required.
- Contain at least 8 characters.
- Contain at least one uppercase letter.

- Contain at least one lowercase letter.
- Contain at least one number.
- Contain at least one special character.

Example of a valid strong password:

```
Student@123
```

The password is not required during normal student editing because the existing Edit operation does not update the password.

2.4 Backend Validation

Server-side validation was implemented in:

```
backend/controllers/studentController.js
```

A reusable validation function was added for both Create and Update operations.

The backend validates:

Field	Backend Validation
Registration Number	Required and valid characters
First Name	Required, minimum 3 characters, valid characters
Last Name	Required, minimum 3 characters, valid characters
Email	Required and valid email format
Phone Number	Required, numeric only, exactly 10 digits
Password	Required during creation and strong password rules

If validation fails, the backend returns HTTP status:

```
400 Bad Request
```

with a clear validation message.

This prevents an administrator or another client from bypassing frontend validation by directly sending invalid data to the API.

2.5 Password Security

The existing password hashing mechanism was preserved.

Student passwords are hashed using:

```
bcrypt.hash(password, 10)
```

Passwords are not stored as plain text in the database.

The implementation does not change the existing authentication mechanism or password hashing process.

2.6 Add and Edit Behaviour

Add Student

During student creation:

1. Administrator opens Add Student.
2. Student information is entered.
3. Client-side validation is performed.
4. If validation fails, the API request is not sent.
5. If validation succeeds, the student data is sent to the backend.
6. Backend validation is performed again.
7. Password is securely hashed.
8. Student account and profile are created.

Edit Student

During student editing:

1. Administrator opens Edit Student.
2. Existing student information is displayed.
3. Administrator modifies the required fields.
4. Client-side validation is performed.
5. Invalid data is rejected.
6. Valid data is sent to the backend.
7. Backend validation is performed again.
8. Existing student record is updated.

Password validation is not required during Edit because the existing Edit API does not modify the student's password.

2.7 Existing CRUD Preservation

Existing CRUD functionality was preserved.

The implementation continues to use:

`student.student_id`

for identifying the correct student.

Validation only controls whether the submitted information is valid.

It does not change:

- Student identity.
- Edit functionality.
- Delete functionality.
- Student database structure.
- Existing API routes.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through manual frontend and backend/API testing.

Testing Types

- Functional testing
- Positive testing
- Negative testing
- Validation testing
- UI testing
- Backend/API testing
- Security testing
- Regression testing
- CRUD verification
- Authentication and authorization verification

Environment

Field	Details
Environment	Development
Frontend	React + Vite
Backend	Node.js + Express
Database	MySQL

Browser	Google Chrome
Test Role	Administrator
API Testing	Postman

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Submit form with empty required fields	Validation message displayed and submission prevented	Required field validation displayed	Pass
TC-02	Enter registration number	Valid registration number accepted	Valid value accepted	Pass
TC-03	Enter invalid registration number	Invalid registration number rejected	Validation message displayed	Pass
TC-04	Enter first name with fewer than 3 characters	First name rejected	Validation message displayed	Pass
TC-05	Enter valid first name with 3+ characters	First name accepted	Valid value accepted	Pass
TC-06	Enter invalid characters in first name	Invalid name rejected	Validation message displayed	Pass
TC-07	Enter last name with fewer than 3 characters	Last name rejected	Validation message displayed	Pass
TC-08	Enter valid last name	Last name accepted	Valid value accepted	Pass
TC-09	Enter invalid characters in last name	Invalid name rejected	Validation message displayed	Pass
TC-10	Enter invalid email	Email rejected	Validation message displayed	Pass
TC-11	Enter valid email	Email accepted	Valid email accepted	Pass
TC-12	Enter phone with fewer than 10 digits	Phone rejected	Validation message displayed	Pass
TC-13	Enter phone with exactly 10 digits	Phone accepted	Valid phone accepted	Pass
TC-14	Enter phone with more than 10 digits	Phone rejected	Validation/restriction applied	Pass
TC-15	Enter letters in phone number	Phone rejected	Validation message displayed	Pass
TC-16	Create student without password	Password rejected	Validation message displayed	Pass

TC-17	Password shorter than 8 characters	Password rejected	Validation message displayed	Pass
TC-18	Password without uppercase letter	Password rejected	Validation message displayed	Pass
TC-19	Password without lowercase letter	Password rejected	Validation message displayed	Pass
TC-20	Password without number	Password rejected	Validation message displayed	Pass
TC-21	Password without special character	Password rejected	Validation message displayed	Pass
TC-22	Enter valid strong password	Password accepted	Valid password accepted	Pass
TC-23	Create student with valid data	Student successfully created	Student created	Pass
TC-24	Create student with invalid data	Student not created	Invalid data rejected	Pass
TC-25	Edit student with invalid data	Update rejected	Validation message displayed	Pass
TC-26	Edit student with valid data	Student successfully updated	Student updated	Pass
TC-27	Direct API request with invalid data	Backend returns HTTP 400	Invalid request rejected	Pass
TC-28	Edit existing student	Correct student updated	Correct `student_id` affected	Pass
TC-29	Delete existing student	Correct student deleted	Existing Delete works	Pass
TC-30	Administrator authentication	Only authenticated administrator can access operation	Authentication works	Pass
TC-31	Administrator authorization	Unauthorized roles cannot modify students	Authorization works	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	31
Passed	31
Failed	0
Pass Rate	100%

Overall Test Result

PASS

The implemented validation functionality successfully passed the defined functional, validation, backend, security, and CRUD verification tests.

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Administrators cannot submit the form when required fields are empty.	Empty field validation test	Met
AC-002	First names containing fewer than 3 characters are rejected.	First name validation test	Met
AC-003	Last names containing fewer than 3 characters are rejected.	Last name validation test	Met
AC-004	Names containing invalid numbers or inappropriate special characters are rejected.	Invalid name test	Met
AC-005	Invalid email addresses are rejected.	Email validation test	Met
AC-006	Phone numbers containing fewer than 10 digits are rejected.	9-digit phone test	Met
AC-007	Phone numbers containing more than 10 digits are rejected.	11-digit phone test	Met
AC-008	Phone numbers containing non-numeric characters are rejected.	Alphabetic phone test	Met
AC-009	A valid 10-digit phone number is accepted.	Valid phone test	Met
AC-010	Weak passwords are rejected when creating a new student.	Weak password tests	Met
AC-011	Strong password containing 8+ characters, uppercase, lowercase, number and special character is accepted.	Strong password test	Met
AC-012	A valid student can be successfully created.	Valid Add Student test	Met
AC-013	Invalid student information cannot be successfully created.	Invalid Add Student test	Met
AC-014	Invalid student information cannot be successfully updated.	Invalid Edit Student test	Met
AC-015	A valid student can be successfully updated.	Valid Edit Student test	Met
AC-016	Direct API requests containing invalid data are rejected by the backend.	Postman API validation test	Met
AC-017	Existing Edit and Delete functionality continues to affect the correct student.	CRUD regression test	Met
AC-018	Existing administrator authentication and authorization continue to work correctly.	Authentication/authorization test	Met

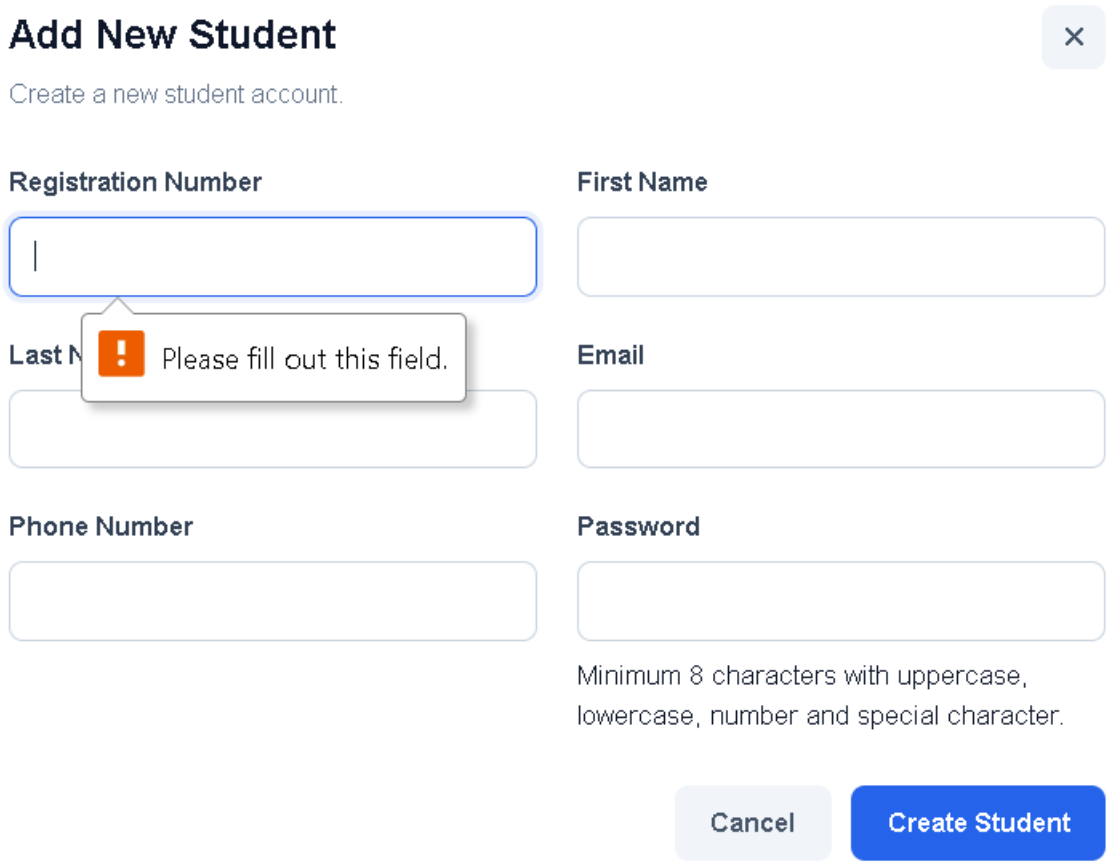
5. Evidence

A maximum of two relevant screenshots should be attached where possible, while additional screenshots may be retained as supporting evidence if required by the examiner.

Evidence 01 – Student Input Validation

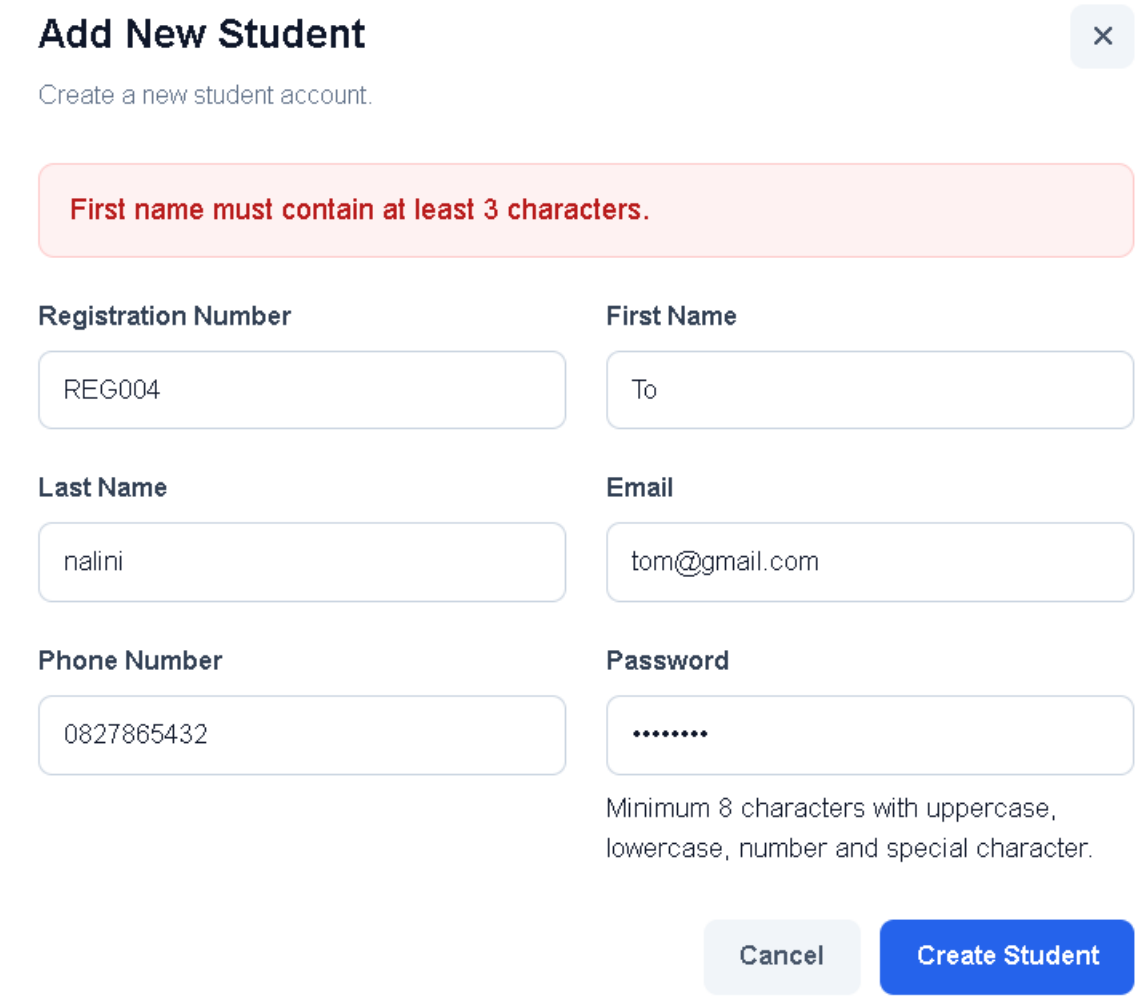
Related: AC-001, AC-002, AC-003, AC-004, AC-005, AC-006, AC-007, AC-008, AC-010, AC-011

Figure 1: Administrator Add/Edit Student form showing validation error messages for invalid student information such as short name, invalid email, invalid phone number, or weak password.

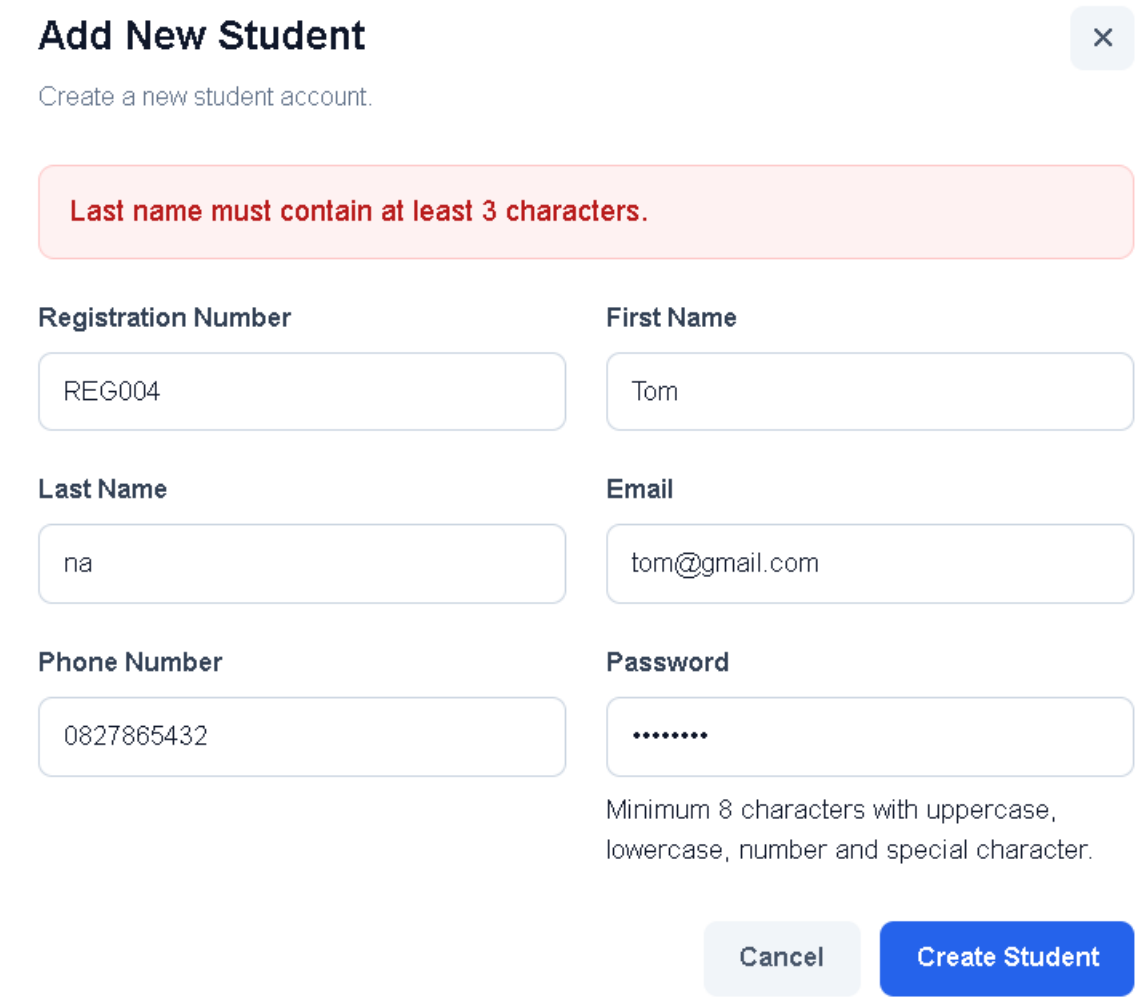


Empty fields

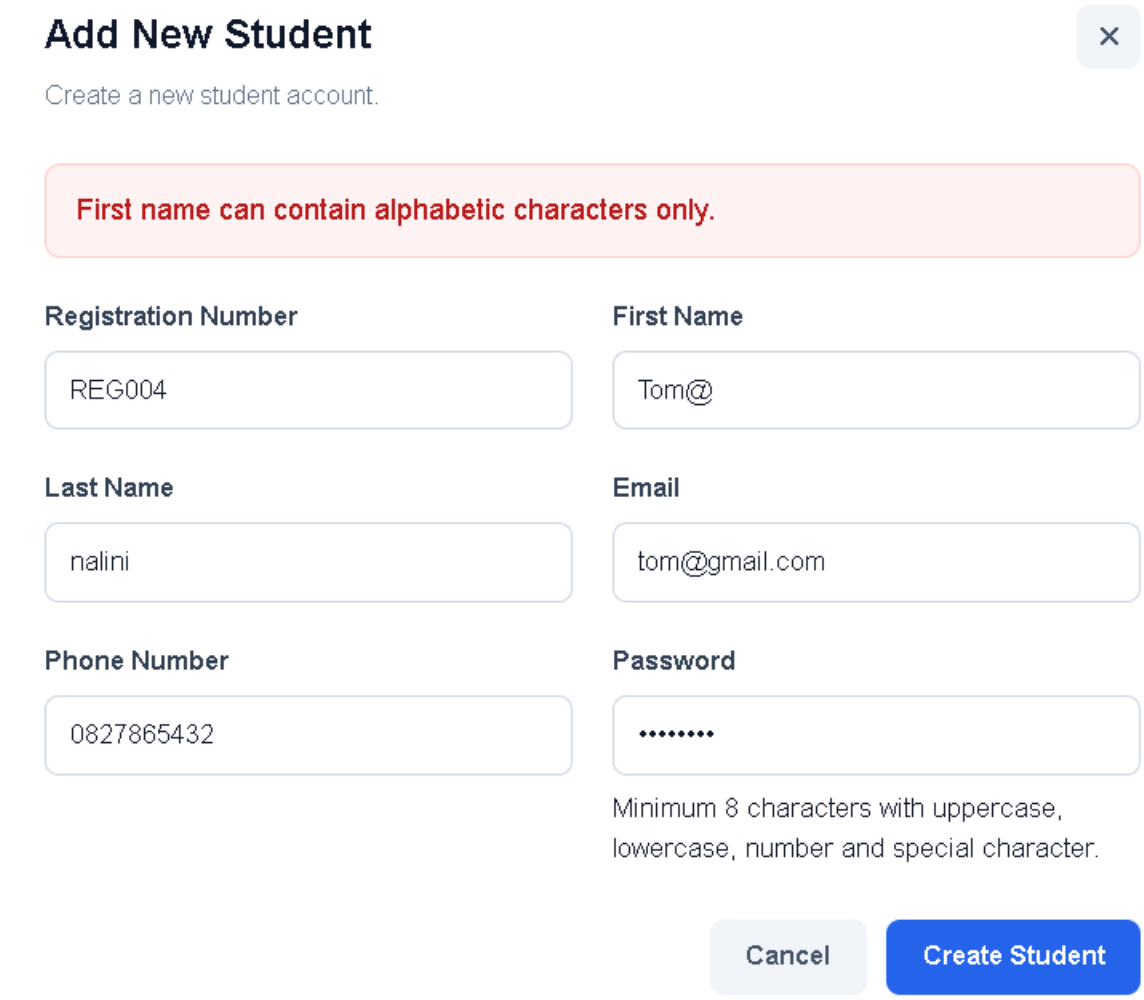
First Name < 3



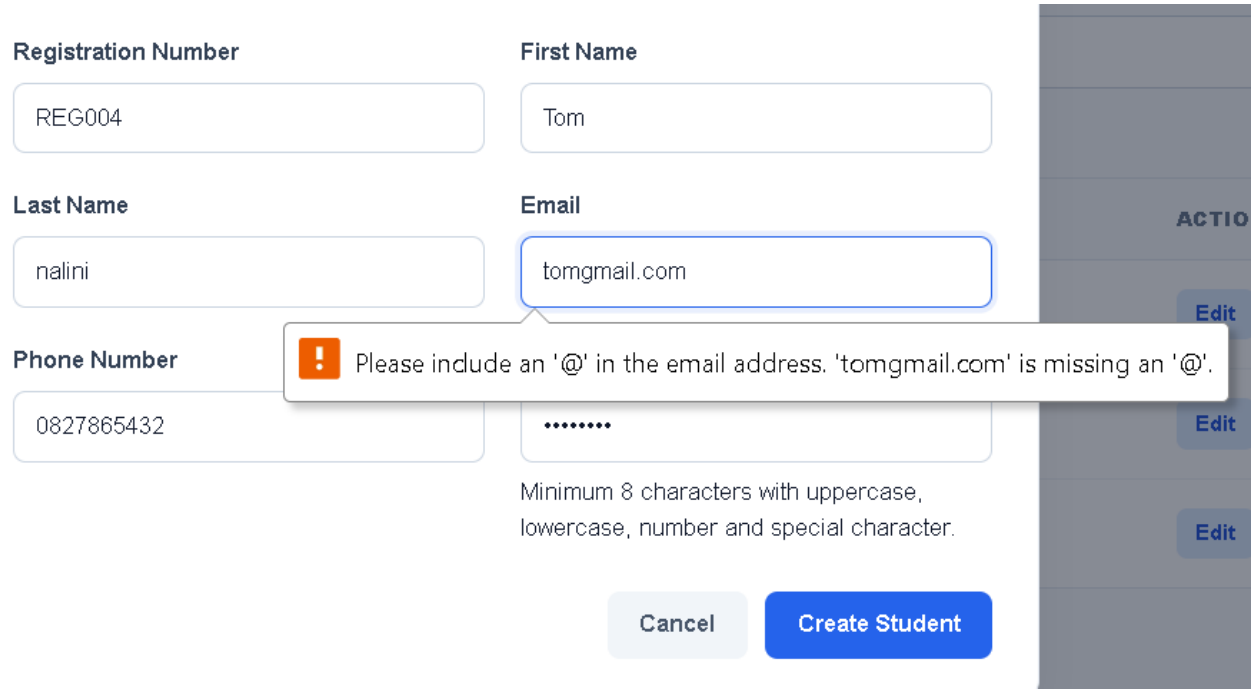
Last Name < 3



Invalid name



Invalid email



Phone < 10 digits

Add New Student

Create a new student account.

Phone number must contain exactly 10 digits.

Registration Number

REG004

First Name

Tom

Last Name

nalini

Email

tom@gmail.com

Phone Number

082786543

Password

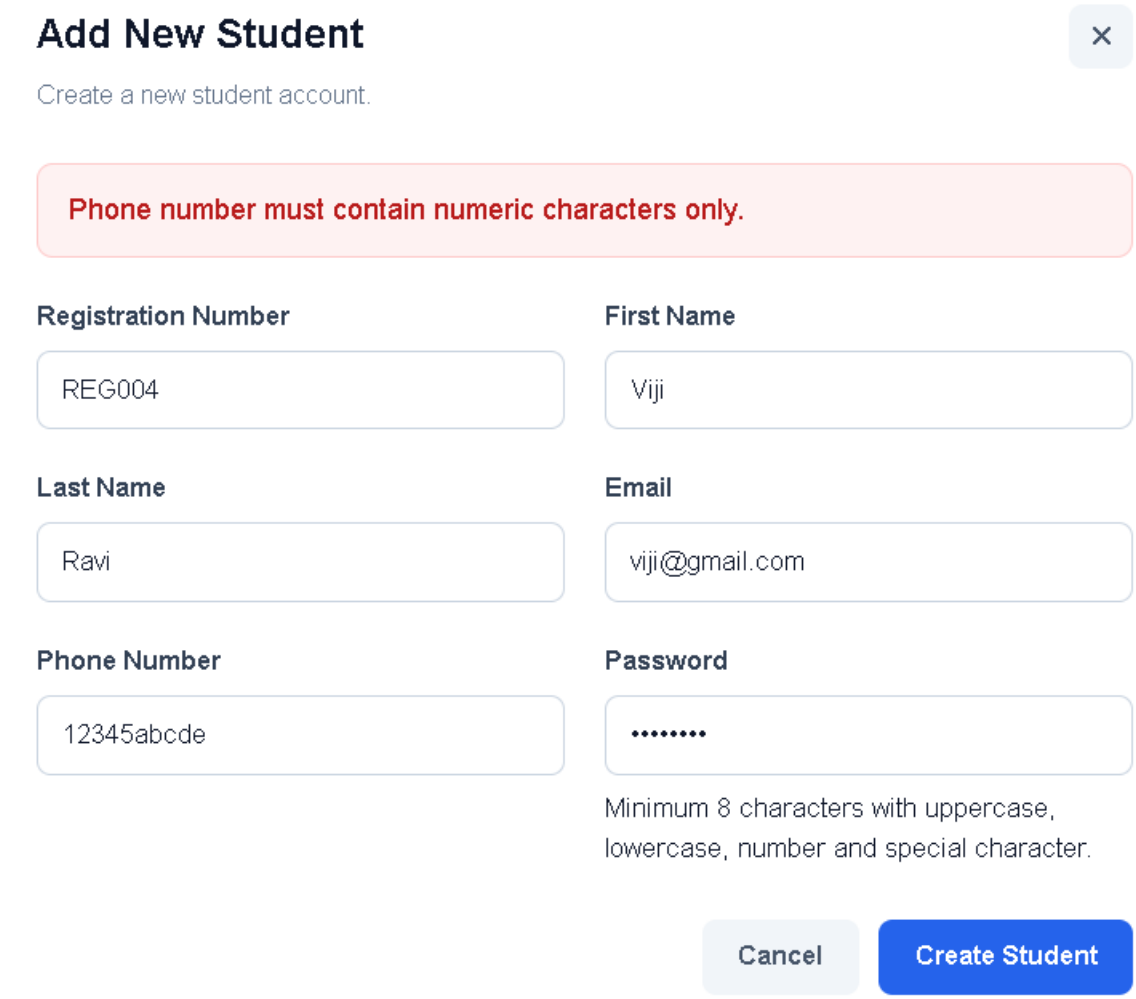
••••••••

Minimum 8 characters with uppercase, lowercase, number and special character.

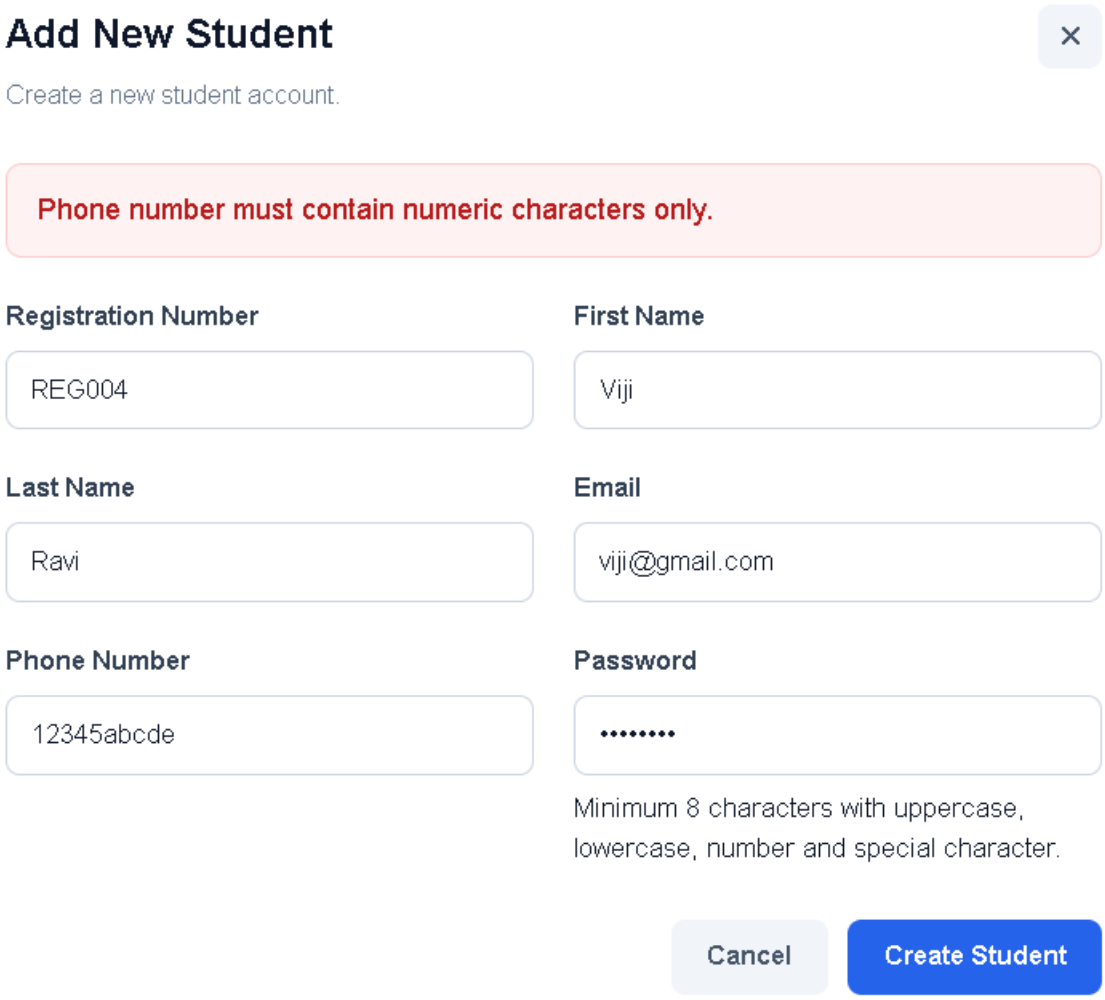
Cancel

Create Student

Phone letters



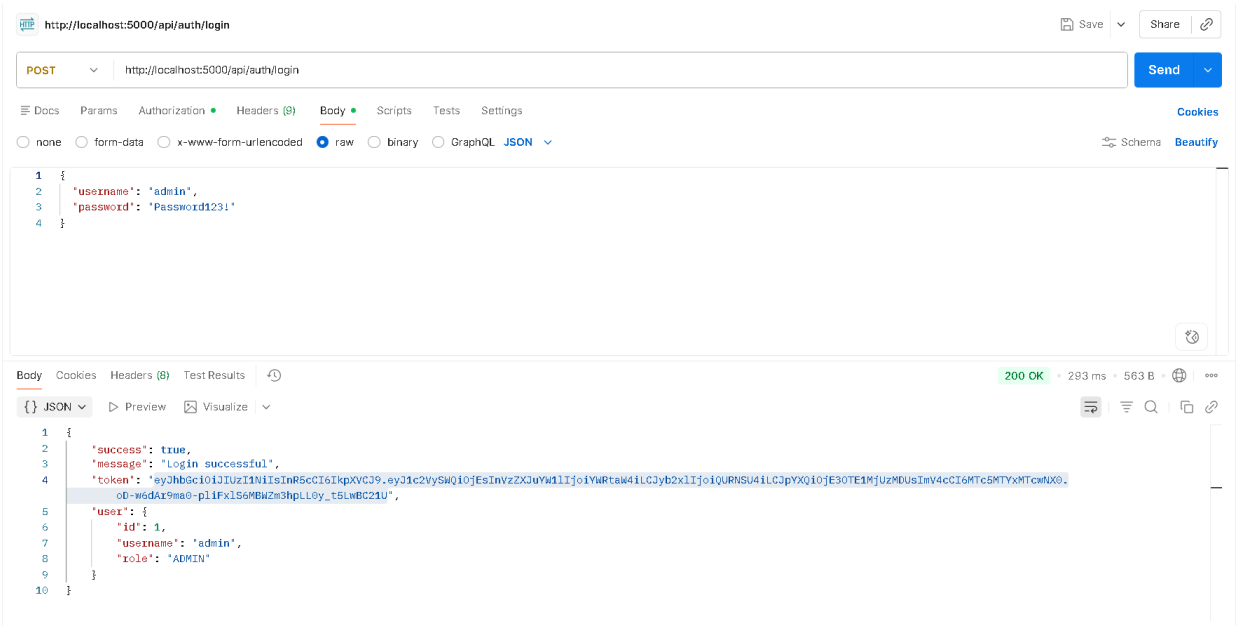
phone number

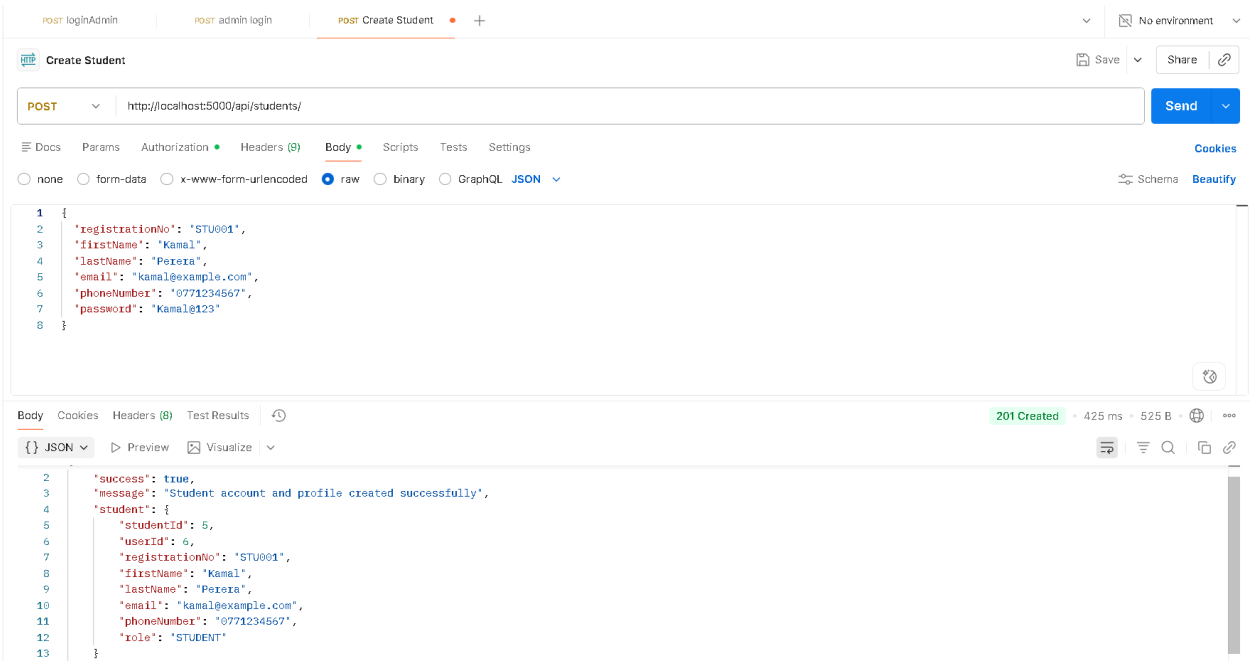


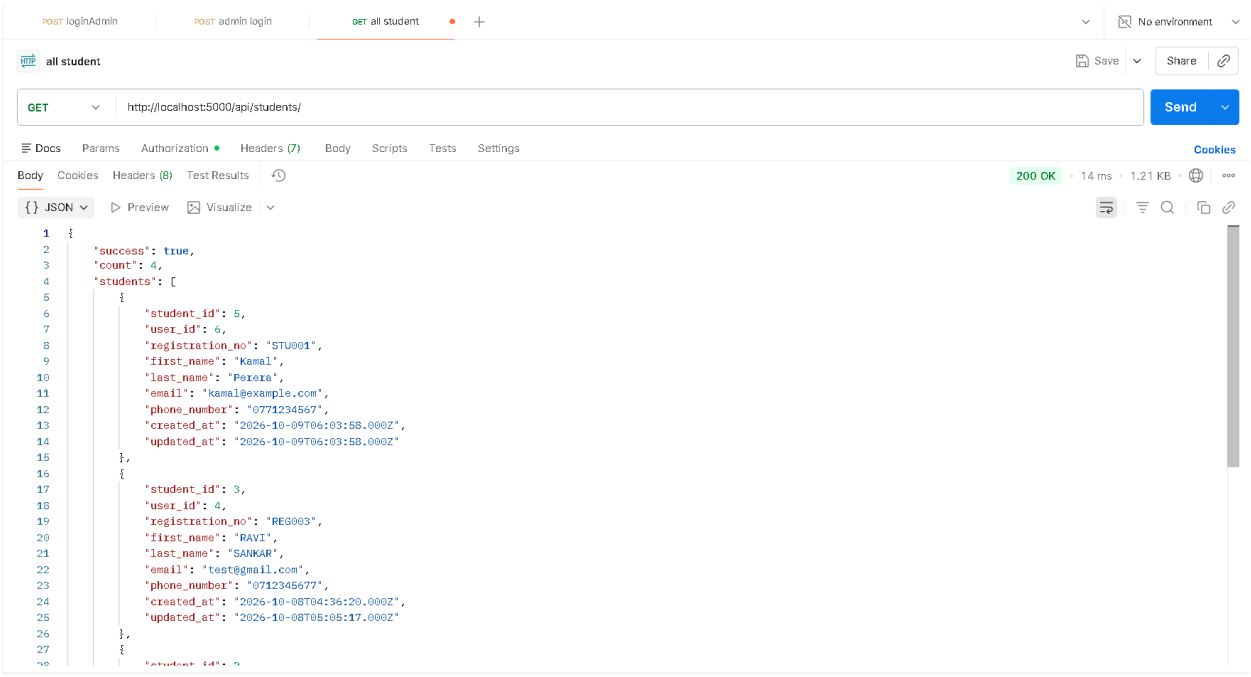
Evidence 02 – Valid Student Creation / Update and Backend Validation

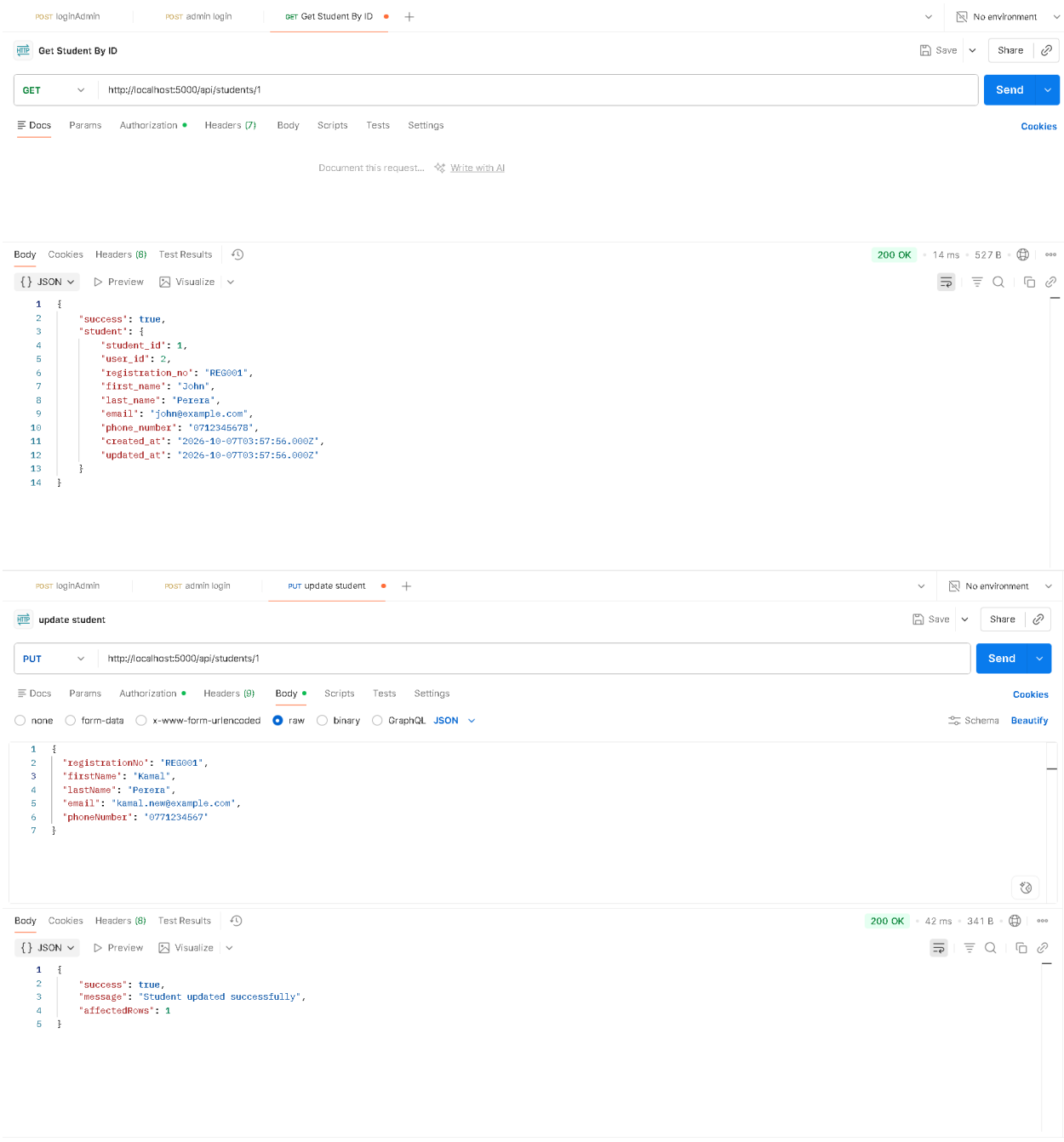
Related: AC-012, AC-014, AC-015, AC-016, AC-017, AC-018

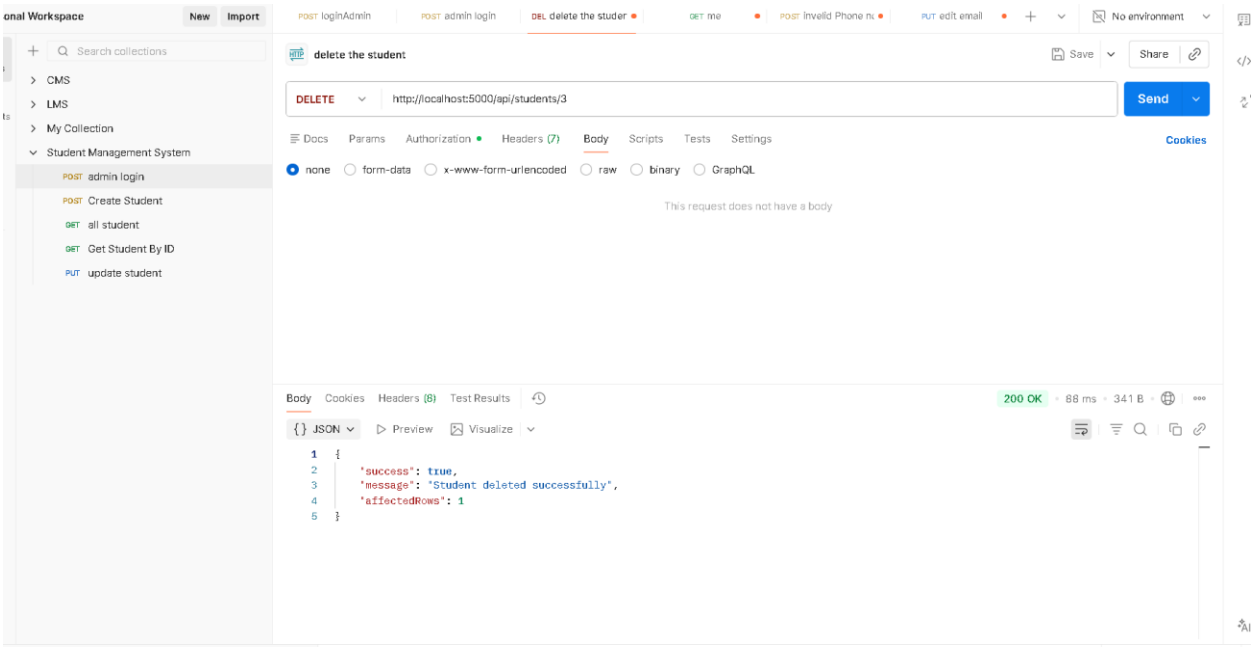
Figure 2: Successful student creation/update and Postman evidence showing that an invalid direct API request is rejected by the backend.

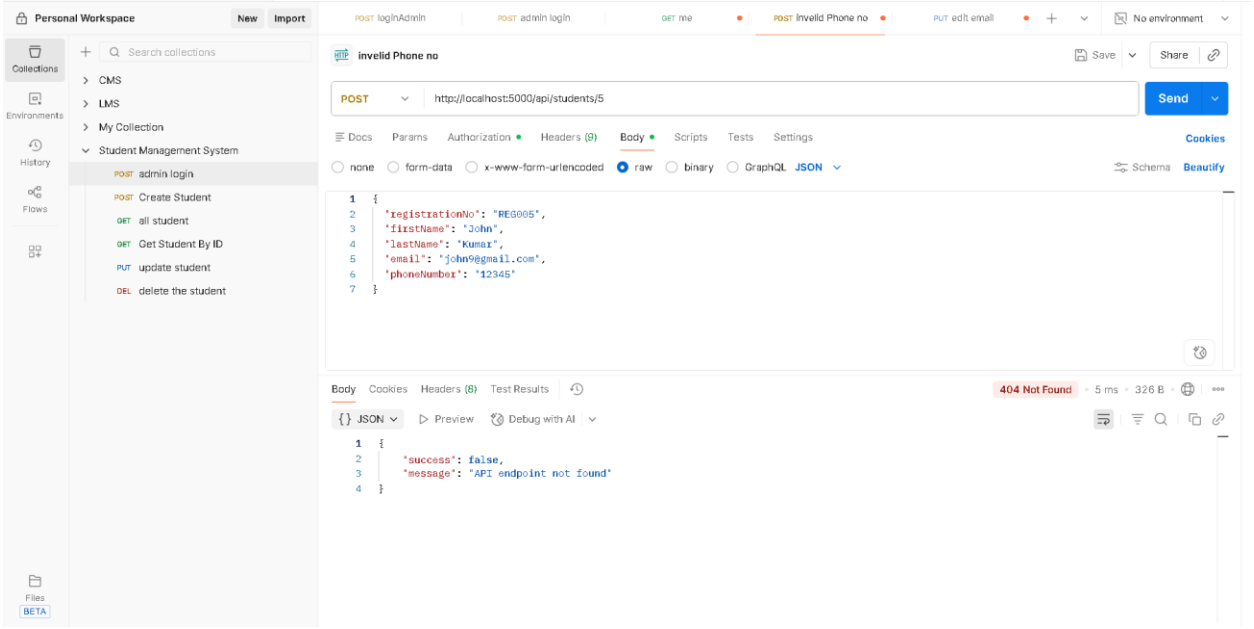

http://localhost:5000/api/auth/login
Save
Share
POST
http://localhost:5000/api/auth/login
Send
Docs Params Authorization Headers (9) Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"username": "admin",
"password": "Password123!"
}
Body Cookies Headers (8) Test Results
200 OK 293 ms 563 B
{} JSON Preview Visualize
{
"success": true,
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjEsInVzZXJuYW1lIjoiYWRtaW4iLCJyb2xlIjoiQURNSU4iLCJpYXQiOjE3OTE1MjUzMDUsImV4cCI6MTc5MTYxMTcwNX0.
oD-w6dAr9ma0-pliFxlS6MBWZm3hpLL0y_t5LwBC21U",
"user": {
"id": 1,
"username": "admin",
"role": "ADMIN"
}
}

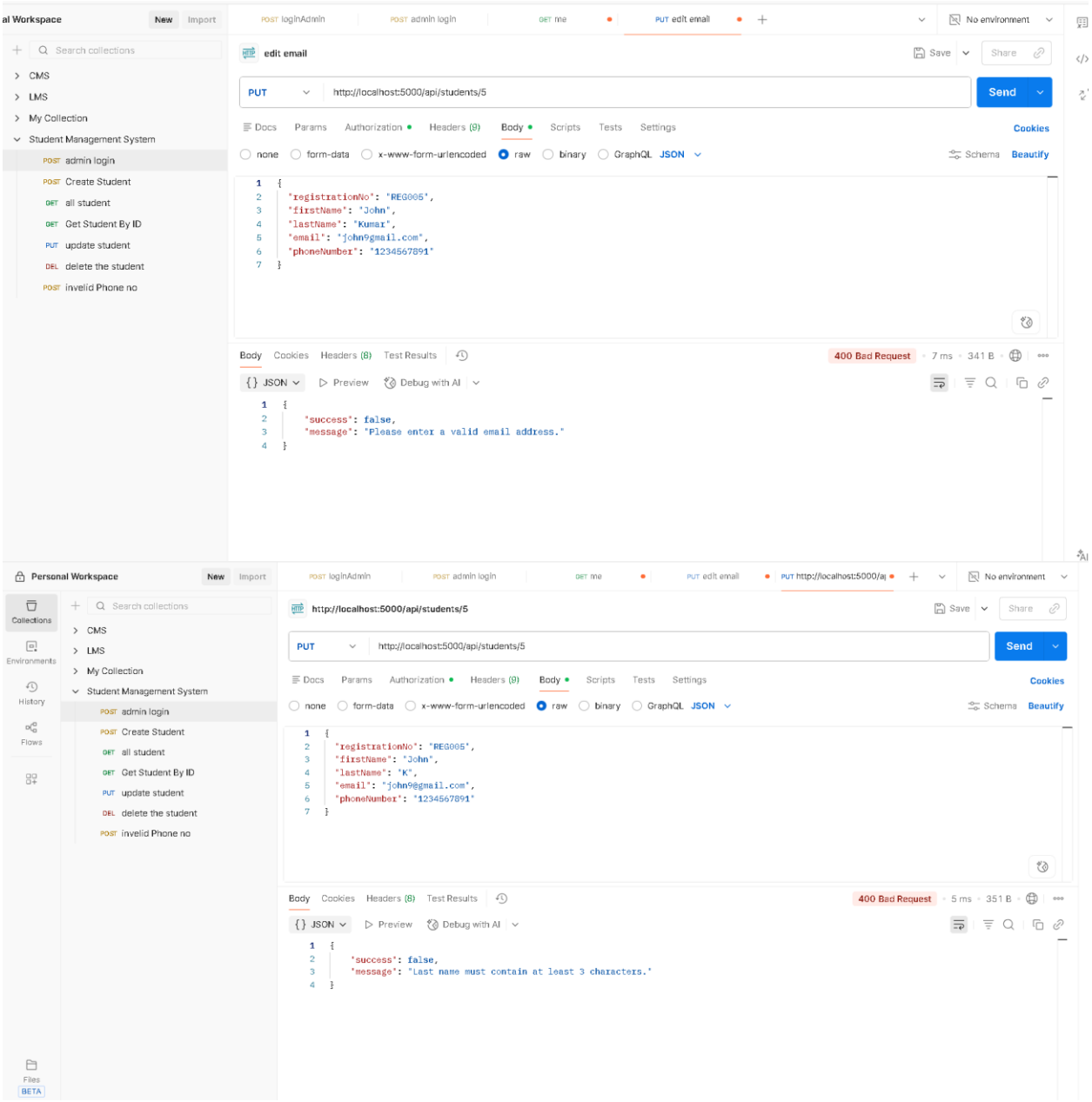

POST loginAdmin
POST admin login
POST Create Student
No environment
Create Student
Save
Share
POST
http://localhost:5000/api/students/
Send
Docs Params Authorization Headers (9) Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"registrationNo": "STU001",
"firstName": "Kamal",
"lastName": "Perera",
"email": "kamal@example.com",
"phoneNumber": "0771234567",
"password": "Kamal@123"
}
Body Cookies Headers (8) Test Results
201 Created 425 ms 525 B
{} JSON Preview Visualize
"success": true,
"message": "Student account and profile created successfully",
"student": {
"studentId": 5,
"userId": 6,
"registrationNo": "STU001",
"firstName": "Kamal",
"lastName": "Perera",
"email": "kamal@example.com",
"phoneNumber": "0771234567",
"role": "STUDENT"
}


POST loginAdmin
POST admin login
GET all student
No environment
all student
Save
Share
GET
http://localhost:5000/api/students/
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Body
Cookies
Headers (8)
Test Results
200 OK
14 ms
1.21 KB
JSON
Preview
Visualize
{
"success": true,
"count": 4,
"students": [
{
"student_id": 5,
"user_id": 6,
"registration_no": "STU001",
"first_name": "Kamal",
"last_name": "Perera",
"email": "kamal@example.com",
"phone_number": "0771234567",
"created_at": "2026-10-09T06:03:58.000Z",
"updated_at": "2026-10-09T06:03:58.000Z"
},
{
"student_id": 3,
"user_id": 4,
"registration_no": "REG003",
"first_name": "RAVI",
"last_name": "SANKAR",
"email": "test@gmail.com",
"phone_number": "0712345677",
"created_at": "2026-10-08T04:36:20.000Z",
"updated_at": "2026-10-08T05:05:17.000Z"
},
{


POST loginAdmin
POST admin login
GET Get Student By ID
No environment
Get Student By ID
Save
Share
GET
http://localhost:5000/api/students/1
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Document this request...
Write with AI
Body
Cookies
Headers (8)
Test Results
200 OK
14 ms
527 B
{} JSON
Preview
Visualize
{
"success": true,
"student": {
"student_id": 1,
"user_id": 2,
"registration_no": "REG001",
"first_name": "John",
"last_name": "Perera",
"email": "john@example.com",
"phone_number": "0712345678",
"created_at": "2026-10-07T03:57:56.000Z",
"updated_at": "2026-10-07T03:57:56.000Z"
}
}
POST loginAdmin
POST admin login
PUT update student
No environment
update student
Save
Share
PUT
http://localhost:5000/api/students/1
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"registrationNo": "REG001",
"firstName": "Kamal",
"lastName": "Perera",
"email": "kamal.new@example.com",
"phoneNumber": "0771234567"
}
Body
Cookies
Headers (8)
Test Results
200 OK
42 ms
341 B
{} JSON
Preview
Visualize
{
"success": true,
"message": "Student updated successfully",
"affectedRows": 1
}


onal Workspace
New
Import
POST loginAdmin
POST admin login
DEL delete the studer
GET me
POST invelid Phone nu
PUT edit email
No environment
Search collections
CMS
LMS
My Collection
Student Management System
POST admin login
POST Create Student
GET all student
GET Get Student By ID
PUT update student
delete the student
Save
Share
DELETE
http://localhost:5000/api/students/3
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
This request does not have a body
Body
Cookies
Headers (8)
Test Results
200 OK
88 ms
341 B
JSON
Preview
Visualize
{
"success": true,
"message": "Student deleted successfully",
"affectedRows": 1
}


Personal Workspace
New
Import
POST loginAdmin
POST admin login
GET me
POST invelid Phone no
PUT edit email
No environment
Collections
Environments
History
Flows
Files
BETA
Search collections
CMS
LMS
My Collection
Student Management System
POST admin login
POST Create Student
GET all student
GET Get Student By ID
PUT update student
DEL delete the student
invelid Phone no
Save
Share
POST
http://localhost:5000/api/students/5
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
'registrationNo': 'REG005',
'firstName': 'John',
'lastName': 'Kumar',
'email': 'john9@gmail.com',
'phoneNumber': '12345'
}
Body
Cookies
Headers (8)
Test Results
404 Not Found
5 ms
326 B
JSON
Preview
Debug with AI
{
"success": false,
"message": "API endpoint not found"
}



6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Existing CRUD	Verified
Authentication	Verified
Authorization	Verified

Password Hashing	Verified
Database Schema Changes	None
New API Endpoints	None
Final Status	Accepted

6.1 Final Summary

CR-004 was successfully implemented as a comprehensive student input validation enhancement for the administrator Add and Edit operations.

Client-side validation was implemented in `StudentFormModal.jsx` to validate student information before submission. The implemented validation includes registration number validation, minimum 3-character first and last names, valid name characters, email format validation, exactly 10-digit numeric phone numbers, and strong password validation for new student accounts.

Server-side validation was implemented in `studentController.js` to ensure that invalid information cannot bypass the frontend validation through direct API requests.

The strong password policy requires at least 8 characters, one uppercase letter, one lowercase letter, one number, and one special character.

The existing password hashing mechanism using bcrypt was preserved, and passwords continue to be stored as hashed values rather than plain text.

No database schema changes or new API endpoints were required.

Existing administrator authentication and authorization were preserved. Existing Add, Edit, and Delete functionality also continued to work using the correct `student_id`.

The implementation was manually tested using frontend validation and backend/API testing. All defined acceptance criteria were verified successfully.

6.2 Issues / Limitations

No significant issues were identified during implementation and testing.

The current implementation performs validation at both client and server levels. Database schema changes were not required because the validation rules are enforced by the application layer.

The password is required during new student creation but is not modified during the existing Edit Student operation.

Student Name: T.Lakshika

Student ID: JF/ICT/24/32

Date: 8 October 2026

Signature: T.Lakshika

~ END OF REPORT ~